

Problem 5: Clustering Palmer Penguins

This notebook implements every part of Problem 5 from HW3 using `penguins.csv`. The clustering code below implements Lloyd's k-means algorithm from scratch, and scikit-learn is only used for the silhouette score requested in part (f).

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import silhouette_score

sns.set_theme(style="whitegrid", context="notebook")
plt.rcParams["figure.figsize"] = (7, 5)
plt.rcParams["axes.spines.top"] = False
plt.rcParams["axes.spines.right"] = False

FEATURES = ["bill_length_mm", "bill_depth_mm", "flipper_length_mm", "body_mass_g"]
df = pd.read_csv("penguins.csv")
```

(a) Exploratory Data Analysis

(a)(i) Basics

```
In [2]: print("Shape:", df.shape)
display(df.describe())

print("Penguins per species:")
display(df["species"].value_counts().rename("count").to_frame())

print("Penguins per island:")
display(df["island"].value_counts().rename("count").to_frame())
```

Shape: (344, 8)

	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	year
count	342.000000	342.000000	342.000000	342.000000	344.000000
mean	43.921930	17.151170	200.915205	4201.754386	2008.029070
std	5.459584	1.974793	14.061714	801.954536	0.818356
min	32.100000	13.100000	172.000000	2700.000000	2007.000000
25%	39.225000	15.600000	190.000000	3550.000000	2007.000000
50%	44.450000	17.300000	197.000000	4050.000000	2008.000000
75%	48.500000	18.700000	213.000000	4750.000000	2009.000000
max	59.600000	21.500000	231.000000	6300.000000	2009.000000

Penguins per species:

	count
species	
Adelie	152
Gentoo	124
Chinstrap	68

Penguins per island:

	count
island	
Biscoe	168
Dream	124
Torgersen	52

(a)(ii) Missing values

The assignment asks us to drop rows with any missing value in the four numerical morphological features.

```
In [3]: missing_by_column = df.isna().sum().rename("missing_values").to_frame()
display(missing_by_column)

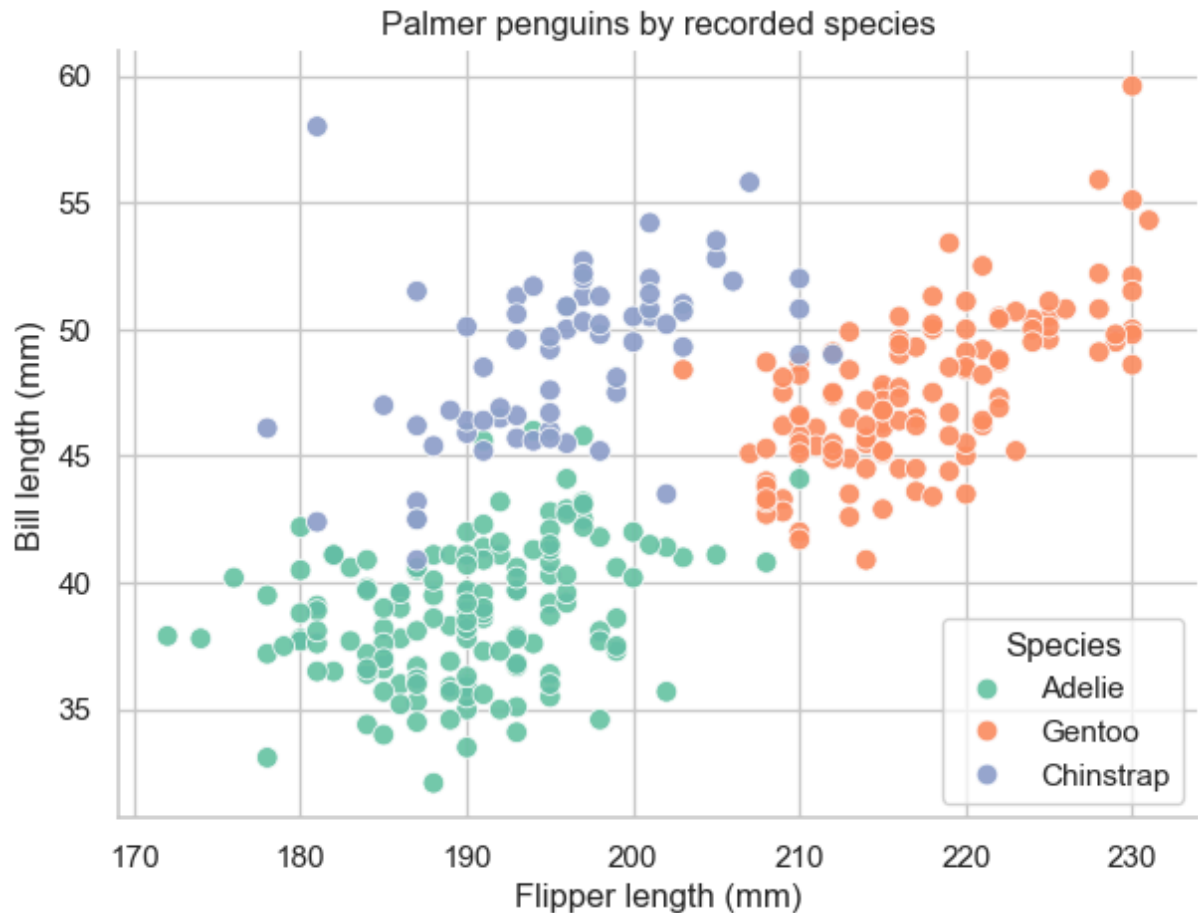
penguins = df.dropna(subset=FEATURES).copy().reset_index(drop=True)
print(f"Rows remaining after dropping NaN values in {FEATURES}: {len(penguins)}")
```

missing_values	
species	0
island	0
bill_length_mm	2
bill_depth_mm	2
flipper_length_mm	2
body_mass_g	2
sex	11
year	0

Rows remaining after dropping NaN values in ['bill_length_mm', 'bill_depth_mm', 'flipper_length_mm', 'body_mass_g']: 342

(a)(iii) Visible clusters

```
In [4]: fig, ax = plt.subplots(figsize=(7, 5))
sns.scatterplot(
    data=penguins,
    x="flipper_length_mm",
    y="bill_length_mm",
    hue="species",
    palette="Set2",
    s=60,
    alpha=0.9,
    ax=ax,
)
ax.set_title("Palmer penguins by recorded species")
ax.set_xlabel("Flipper length (mm)")
ax.set_ylabel("Bill length (mm)")
ax.legend(title="Species")
plt.show()
```



There are visible groups, but not three perfectly separated blobs in this projection. Gentoo is the easiest to separate because it tends to have much longer flippers, while Adelie and Chinstrap overlap more in the flipper-length / bill-length view.

(b) Preprocessing

(b)(i) Pick the features

```
In [5]: X_raw = penguins[FEATURES].to_numpy(dtype=float)
species = penguins["species"].copy()

print("Feature matrix shape:", X_raw.shape)
display(penguins[FEATURES].head())
```

Feature matrix shape: (342, 4)

	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g
0	39.1	18.7	181.0	3750.0
1	39.5	17.4	186.0	3800.0
2	40.3	18.0	195.0	3250.0
3	36.7	19.3	193.0	3450.0
4	39.3	20.6	190.0	3650.0

(b)(ii) Why standardize

k-means uses Euclidean distance, so features measured on larger numerical scales dominate the objective. Here, body mass is measured in thousands of grams while bill measurements are in millimeters, so without standardization the clustering would mostly optimize body mass and underweight bill and flipper geometry.

(b)(iii) Standardize

```
In [6]: feature_means = X_raw.mean(axis=0)
feature_stds = X_raw.std(axis=0, ddof=0)
X = (X_raw - feature_means) / feature_stds

standardized_summary = pd.DataFrame(
    {
        "mean_after_standardization": X.mean(axis=0),
        "std_after_standardization": X.std(axis=0, ddof=0),
    },
    index=FEATURES,
)
display(standardized_summary)
```

	mean_after_standardization	std_after_standardization
bill_length_mm	1.662088e-16	1.0
bill_depth_mm	4.155221e-16	1.0
flipper_length_mm	-8.310441e-16	1.0
body_mass_g	8.310441e-17	1.0

(c) Implement k-means from scratch

```
In [7]: def kmeans(X, k, max_iter=100, seed=0):
# returns (centroids, labels, inertia)
X = np.asarray(X, dtype=float)
n_samples = X.shape[0]
if not 1 <= k <= n_samples:
```

```

        raise ValueError("k must be between 1 and the number of rows in X")

    rng = np.random.default_rng(seed)
    initial_indices = rng.choice(n_samples, size=k, replace=False)
    centroids = X[initial_indices].copy()

    labels = None
    inertia_history = []

    for _ in range(max_iter):
        squared_distances = ((X[:, None, :] - centroids[None, :, :]) ** 2).sum(axis=2)
        new_labels = squared_distances.argmin(axis=2)

        new_centroids = centroids.copy()
        for cluster_id in range(k):
            cluster_mask = new_labels == cluster_id
            if cluster_mask.any():
                new_centroids[cluster_id] = X[cluster_mask].mean(axis=0)

        inertia = float(((X - new_centroids[new_labels]) ** 2).sum())
        inertia_history.append(inertia)

        if labels is not None and np.array_equal(new_labels, labels):
            labels = new_labels
            centroids = new_centroids
            break

        labels = new_labels
        centroids = new_centroids

    kmeans.inertia_history_ = inertia_history
    return centroids, labels, inertia_history[-1]

def best_kmeans(X, k, seeds=range(10), max_iter=100):
    best = None
    for seed in seeds:
        centroids, labels, inertia = kmeans(X, k, max_iter=max_iter, seed=seed)
        result = {
            "seed": seed,
            "centroids": centroids,
            "labels": labels,
            "inertia": inertia,
            "history": kmeans.inertia_history_.copy(),
        }
        if best is None or inertia < best["inertia"]:
            best = result
    return best

```

(c)(ii) Sanity check on the objective

```

In [8]: _, _, example_inertia = kmeans(X, k=3, seed=0)
        history = np.array(kmeans.inertia_history_)

        print(f"Final inertia for seed 0: {example_inertia:.3f}")

```

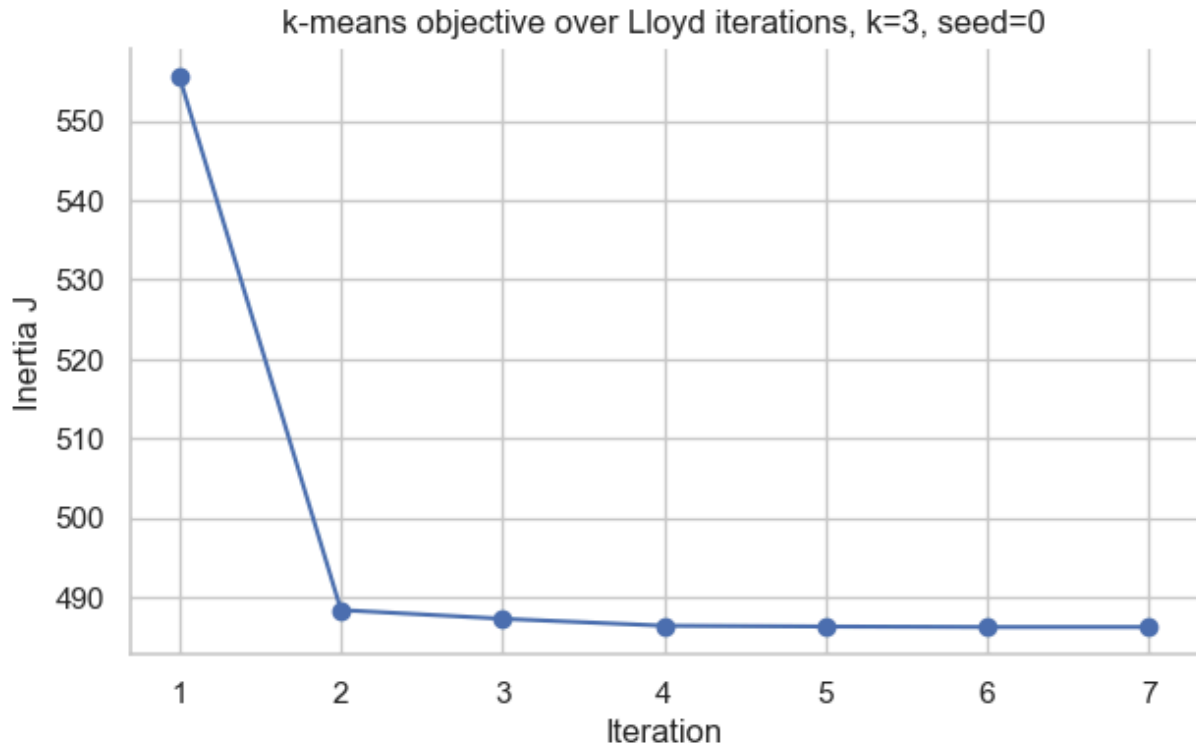
```
print("Inertia history:", np.round(history, 3))
print("Monotone non-increasing:", bool(np.all(np.diff(history) <= 1e-10)))

fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(np.arange(1, len(history) + 1), history, marker="o")
ax.set_title("k-means objective over Lloyd iterations, k=3, seed=0")
ax.set_xlabel("Iteration")
ax.set_ylabel("Inertia J")
plt.show()
```

Final inertia for seed 0: 486.202

Inertia history: [555.582 488.346 487.246 486.369 486.268 486.202 486.202]

Monotone non-increasing: True



The objective is monotone non-increasing on this run, as expected for Lloyd's algorithm.

(d) Cluster and validate

Because k-means can get stuck in a poor local minimum, the main `k=3` clustering below uses the best inertia among seeds 0 through 9. Part (g) studies that initialization sensitivity directly.

```
In [9]: main_run = best_kmeans(X, k=3, seeds=range(10))
centroids = main_run["centroids"]
cluster_labels = main_run["labels"]
final_inertia = main_run["inertia"]

print(f"Best seed among 0..9: {main_run['seed']}")
print(f"Final inertia for k=3: {final_inertia:.3f}")
```

Best seed among 0..9: 2
 Final inertia for k=3: 379.393

(d)(ii) Did we rediscover the species?

```
In [10]: cluster_species_table = pd.crosstab(
    pd.Series(cluster_labels, name="cluster"),
    species.rename("species"),
)
purity = cluster_species_table.max(axis=1).sum() / cluster_species_table.to_numpy()

display(cluster_species_table)
print(f"Clustering purity: {purity:.3f}")
```

species **Adelie** **Chinstrap** **Gentoo**

cluster

0	24	63	0
1	127	5	0
2	0	0	123

Clustering purity: 0.915

The clustering strongly recovers Gentoo, and it partially separates Adelie from Chinstrap. The main confusion is between Adelie and Chinstrap, which matches the species-colored scatter plot from part (a).

(d)(iii) Cluster profiles in original units

```
In [11]: centroids_original_units = centroids * feature_stds + feature_means
centroid_table = pd.DataFrame(centroids_original_units, columns=FEATURES)
centroid_table.index.name = "cluster"
display(centroid_table.round(2))
```

	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g
cluster				
0	47.53	18.76	196.90	3902.01
1	38.21	18.11	188.40	3584.66
2	47.50	14.98	217.19	5076.02

Cluster 0 represents penguins with relatively long bills, deep bills, medium flippers, and medium body mass; this profile is closest to Chinstrap.

Cluster 1 represents smaller penguins with shorter bills, deep bills, shorter flippers, and lower body mass; this profile is closest to Adelie.

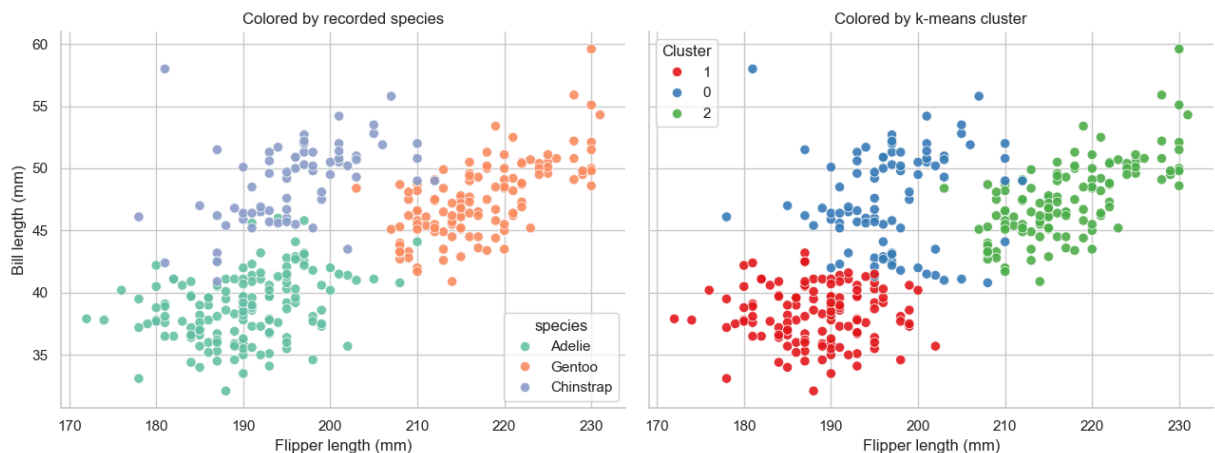
Cluster 2 represents larger penguins with long flippers, high body mass, and shallower bills; this profile is closest to Gentoo.

(d)(iv) Visualize clusters

```
In [12]: plot_df = penguins.copy()
plot_df["cluster"] = cluster_labels.astype(str)

fig, axes = plt.subplots(1, 2, figsize=(13, 5), sharex=True, sharey=True)
sns.scatterplot(
    data=plot_df,
    x="flipper_length_mm",
    y="bill_length_mm",
    hue="species",
    palette="Set2",
    s=55,
    alpha=0.9,
    ax=axes[0],
)
axes[0].set_title("Colored by recorded species")
axes[0].set_xlabel("Flipper length (mm)")
axes[0].set_ylabel("Bill length (mm)")

sns.scatterplot(
    data=plot_df,
    x="flipper_length_mm",
    y="bill_length_mm",
    hue="cluster",
    palette="Set1",
    s=55,
    alpha=0.9,
    ax=axes[1],
)
axes[1].set_title("Colored by k-means cluster")
axes[1].set_xlabel("Flipper length (mm)")
axes[1].set_ylabel("Bill length (mm)")
axes[1].legend(title="Cluster")
plt.tight_layout()
plt.show()
```



The cluster-colored plot looks similar to the species-colored plot for Gentoo, but differs in the Adelie/Chinstrap overlap region. k-means makes a geometric split there, while the species labels are not perfectly separated in this two-dimensional projection.

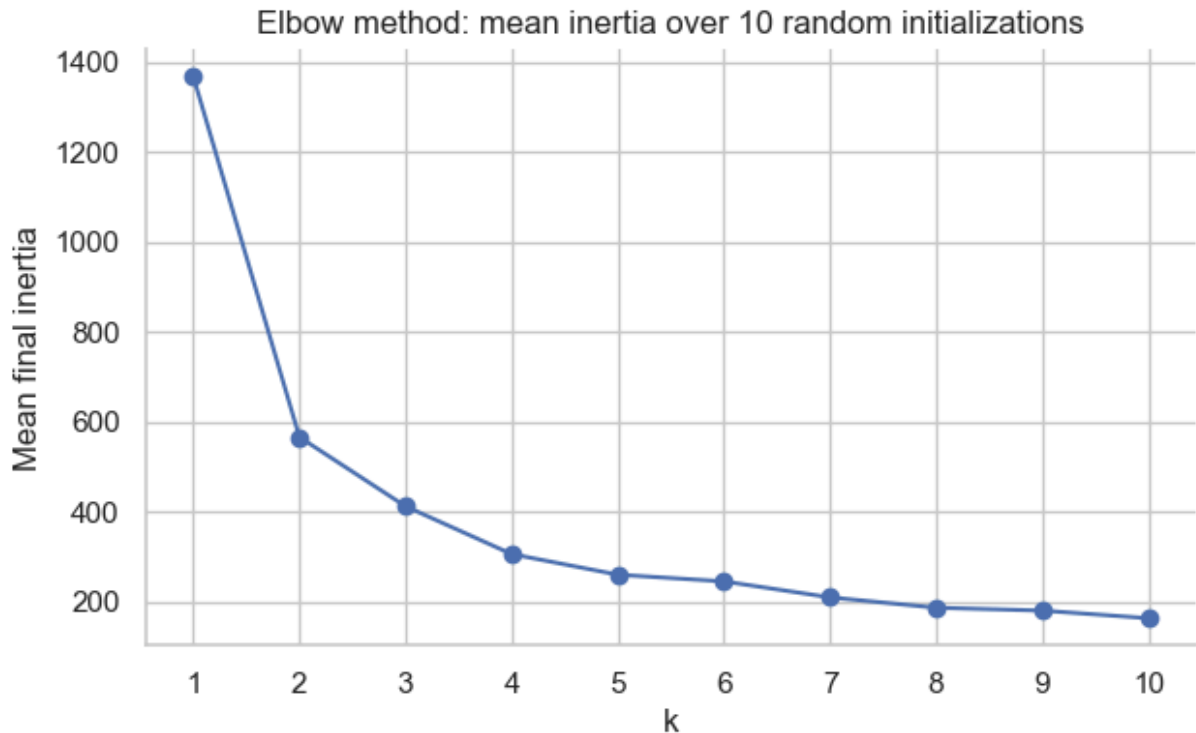
(e) The elbow method

```
In [13]: elbow_rows = []
for k in range(1, 11):
    inertias = [kmeans(X, k=k, seed=seed)[2] for seed in range(10)]
    elbow_rows.append(
        {
            "k": k,
            "mean_inertia": np.mean(inertias),
            "std_inertia": np.std(inertias, ddof=0),
            "min_inertia": np.min(inertias),
        }
    )

elbow_df = pd.DataFrame(elbow_rows)
display(elbow_df.round(3))

fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(elbow_df["k"], elbow_df["mean_inertia"], marker="o")
ax.set_xticks(range(1, 11))
ax.set_title("Elbow method: mean inertia over 10 random initializations")
ax.set_xlabel("k")
ax.set_ylabel("Mean final inertia")
plt.show()
```

	k	mean_inertia	std_inertia	min_inertia
0	1	1368.000	0.000	1368.000
1	2	565.708	0.000	565.708
2	3	411.627	48.865	379.393
3	4	305.269	5.222	300.400
4	5	259.483	27.127	232.597
5	6	244.537	30.338	205.127
6	7	209.003	30.523	187.155
7	8	185.960	22.712	171.368
8	9	179.548	23.879	160.739
9	10	162.613	9.241	152.667



The clearest bend is at $k = 2$: moving from one cluster to two gives a very large drop, and later improvements are more gradual. There is still a smaller improvement at $k = 3$, which is biologically meaningful because the dataset has three recorded species, but the elbow alone says the strongest morphological split is closer to two groups.

(f) Silhouette score

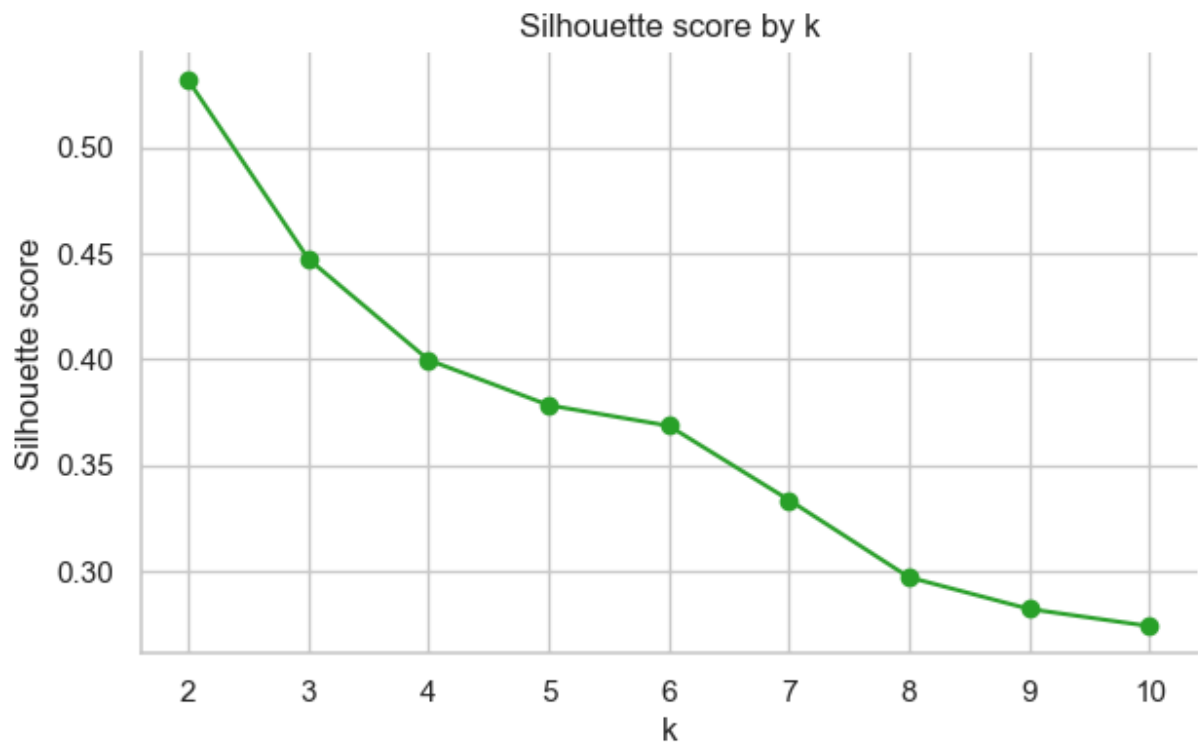
```
In [14]: silhouette_rows = []
for k in range(2, 11):
    run = best_kmeans(X, k=k, seeds=range(10))
    score = silhouette_score(X, run["labels"])
    silhouette_rows.append(
        {
            "k": k,
            "best_seed": run["seed"],
            "best_inertia": run["inertia"],
            "silhouette": score,
        }
    )

silhouette_df = pd.DataFrame(silhouette_rows)
display(silhouette_df.round(3))

fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(silhouette_df["k"], silhouette_df["silhouette"], marker="o", color="tab:green")
ax.set_xticks(range(2, 11))
ax.set_title("Silhouette score by k")
ax.set_xlabel("k")
ax.set_ylabel("Silhouette score")
plt.show()
```

```
best_silhouette_row = silhouette_df.loc[silhouette_df["silhouette"].idxmax()]
print(
    f"Best silhouette occurs at k={int(best_silhouette_row['k'])} "
    f"with score {best_silhouette_row['silhouette']:.3f}."
)
```

	k	best_seed	best_inertia	silhouette
0	2	0	565.708	0.532
1	3	2	379.393	0.447
2	4	1	300.400	0.400
3	5	6	232.597	0.378
4	6	4	205.127	0.369
5	7	6	187.155	0.334
6	8	9	171.368	0.297
7	9	4	160.739	0.282
8	10	1	152.667	0.274



Best silhouette occurs at k=2 with score 0.532.

The silhouette score also prefers `k = 2`, so it agrees with the strongest elbow signal. I would trust that for the unsupervised geometry of these four standardized features: the strongest split is Gentoo versus non-Gentoo, while Adelie and Chinstrap are less cleanly separated by k-means.

(g) Sensitivity to initialization

```
In [15]: seed_rows = []
for seed in range(50):
    centroids_seed, labels_seed, inertia_seed = kmeans(X, k=3, seed=seed)
    seed_rows.append({"seed": seed, "inertia": inertia_seed, "labels": labels_seed})

seed_df = pd.DataFrame(seed_rows)
display(seed_df[["seed", "inertia"]].describe().round(3))

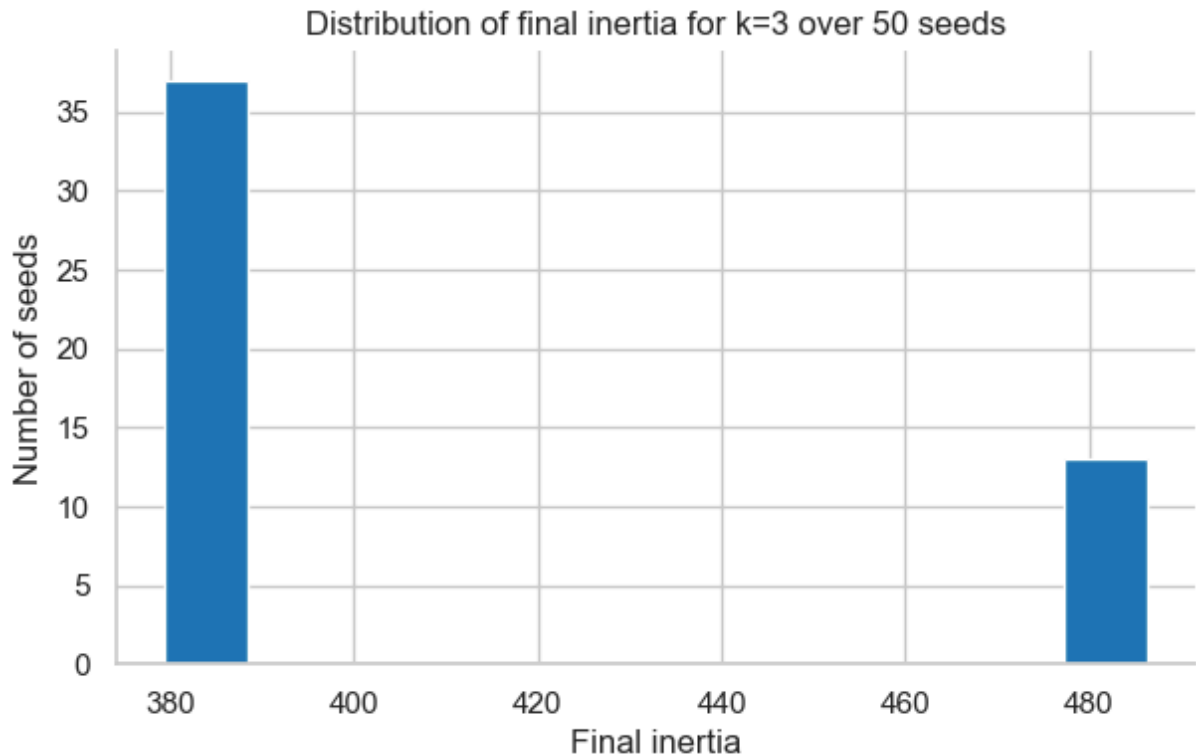
best_seed = int(seed_df.loc[seed_df["inertia"].idxmin(), "seed"])
worst_seed = int(seed_df.loc[seed_df["inertia"].idxmax(), "seed"])
best_seed_inertia = float(seed_df.loc[seed_df["inertia"].idxmin(), "inertia"])
worst_seed_inertia = float(seed_df.loc[seed_df["inertia"].idxmax(), "inertia"])

print(f"Lowest-inertia seed: {best_seed} with inertia {best_seed_inertia:.3f}")
print(f"Highest-inertia seed: {worst_seed} with inertia {worst_seed_inertia:.3f}")
print(f"Spread: {worst_seed_inertia - best_seed_inertia:.3f}")

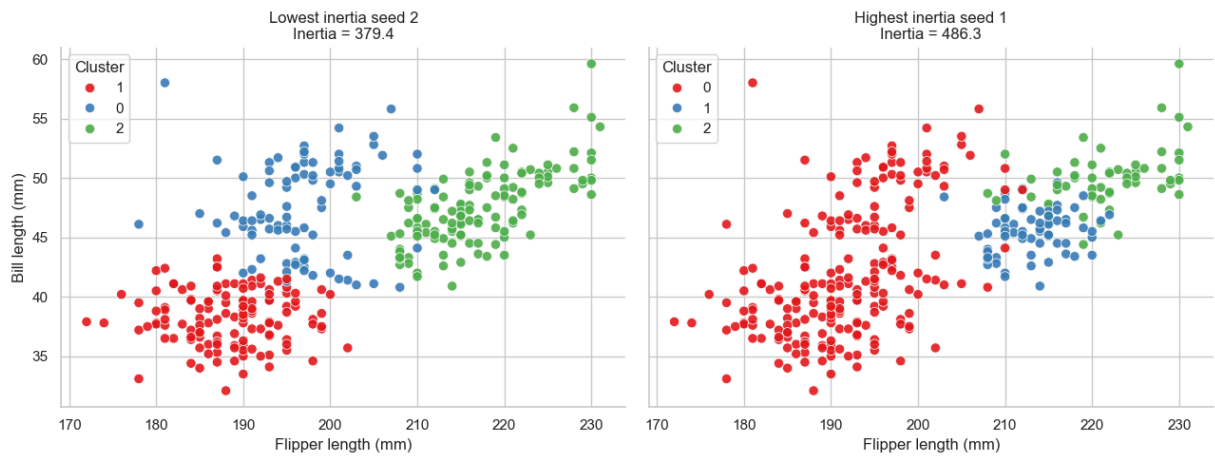
fig, ax = plt.subplots(figsize=(7, 4))
ax.hist(seed_df["inertia"], bins=12, color="tab:blue", edgecolor="white")
ax.set_title("Distribution of final inertia for k=3 over 50 seeds")
ax.set_xlabel("Final inertia")
ax.set_ylabel("Number of seeds")
plt.show()
```

	seed	inertia
count	50.000	50.000
mean	24.500	407.466
std	14.577	47.193
min	0.000	379.393
25%	12.250	379.393
50%	24.500	379.403
75%	36.750	459.985
max	49.000	486.297

Lowest-inertia seed: 2 with inertia 379.393
 Highest-inertia seed: 1 with inertia 486.297
 Spread: 106.905



```
In [16]: fig, axes = plt.subplots(1, 2, figsize=(13, 5), sharex=True, sharey=True)
for ax, seed, title in [
    (axes[0], best_seed, f"Lowest inertia seed {best_seed}"),
    (axes[1], worst_seed, f"Highest inertia seed {worst_seed}"),
]:
    _, labels_seed, inertia_seed = kmeans(X, k=3, seed=seed)
    temp = penguins.copy()
    temp["cluster"] = labels_seed.astype(str)
    sns.scatterplot(
        data=temp,
        x="flipper_length_mm",
        y="bill_length_mm",
        hue="cluster",
        palette="Set1",
        s=55,
        alpha=0.9,
        ax=ax,
    )
    ax.set_title(f"{title}\nInertia = {inertia_seed:.1f}")
    ax.set_xlabel("Flipper length (mm)")
    ax.set_ylabel("Bill length (mm)")
    ax.legend(title="Cluster")
plt.tight_layout()
plt.show()
```



The spread is substantial: the worst seed among the first 50 has much higher inertia than the best seed. The two visualizations disagree more than just a few boundary points; the poor run splits Gentoo into two clusters and merges most Adelie and Chinstrap penguins, while the low-inertia run separates Gentoo and gives a more meaningful Adelie/Chinstrap split.

(h) Summary and recommendation

Using only the four standardized morphological measurements, the strongest unsupervised structure appears to be two groups, not three: both the elbow curve and the silhouette score point most clearly to $k = 2$. A three-cluster solution is still biologically useful because it recovers the recorded species labels with about 91.5% purity when using the best-inertia run among seeds 0 through 9, with Gentoo almost perfectly separated and most remaining confusion between Adelie and Chinstrap. I would report that the data strongly separates Gentoo from the other penguins, while the Adelie/Chinstrap boundary is weaker and more sensitive to initialization. Because Lloyd's algorithm can land in different local minima, any reported k-means result should use multiple random initializations and keep the lowest-inertia run. A limitation is that this analysis uses only four morphological measurements after dropping rows with missing numerical values, so it may miss ecological or genetic information that could distinguish species more reliably.